Stock Management System - Documentation

Stock Management System - Documentation

1. Project Title

Stock Management System

2. System Overview

System Name

Stock Management System

Case Study

A retail business needs to manage its inventory across multiple warehouses. The business must track products, suppliers, stock levels, purchase orders, and order items to ensure efficient and accurate inventory management.

System Overview

The Stock Management System is a web-based application designed to help a retail business manage inventory efficiently. The system will provide a centralized platform for managing product and category information, supplier records, warehouse locations, stock quantities, reorder levels, purchase orders, and order items.

The system will support multiple warehouses and allow authorized users to monitor inventory levels for different products in each warehouse. It will also maintain supplier and purchasing information, allowing users to create and track purchase orders and the products included in those orders.

By storing related information in a relational database, the system will improve data consistency, reduce manual record-keeping errors, and provide users with organized access to inventory and purchasing information.

3. Problem Statement

Retail businesses that manage inventory manually or through separate records may experience several problems, including:

- Product information may be stored in multiple locations, making it difficult to maintain accurate and up-to-date records.
- Stock quantities across different warehouses may become outdated, resulting in overstocking or stock shortages.
- Supplier details and purchase orders may be difficult to track, especially when orders involve multiple products.
- Employees may have difficulty identifying products that have reached or fallen below their reorder levels.
- Manual record keeping increases the risk of duplicate entries, inconsistent data, and human error.
- Managers may not have timely access to inventory and purchasing information needed for decision-making.
- Separate records for products, warehouses, suppliers, stock, and orders can make it difficult to determine relationships between related information.

These problems can lead to inefficient warehouse operations, inaccurate inventory records, delayed purchasing decisions, and increased operational costs.

4. Proposed Solution

The proposed solution is a centralized web-based Stock Management System that stores product, supplier, warehouse, stock, purchase order, and order item information in a relational database.

The system will provide the following capabilities:

- Centralized management of product categories and product information.
- Management of supplier records and supplier contact details.
- Support for multiple warehouses and warehouse managers.
- Tracking of stock quantities for each product in each warehouse.
- Monitoring of reorder levels to identify low-stock items.
- Creation and tracking of purchase orders and individual order items.
- User roles for administrators, inventory managers, and warehouse staff.
- Data validation and database constraints to improve data integrity.
- A user interface that allows authorized users to manage inventory and purchasing records.

By replacing fragmented manual records with a centralized database and web interface, the system will improve inventory visibility, reduce data-entry errors, support more informed purchasing decisions, and make warehouse operations easier to manage.

5. Objectives

5.1 General Objective

To design and develop a web-based Stock Management System that enables a retail business to efficiently manage products, suppliers, warehouses, inventory levels, and purchase orders through a centralized relational database.

5.2 Specific Objectives

- To create a system for managing product categories and product information, including product names, SKUs, descriptions, and prices.
- To implement supplier management for storing supplier contact and business information.
- To support multiple warehouses and allow authorized users to manage warehouse details and assigned managers.
- To track stock quantities for each product within each warehouse.
- To monitor reorder levels and identify products with low inventory.
- To implement purchase order management, including order details, suppliers, warehouses, dates, order status, and individual order items.
- To design a relational database that represents products, suppliers, warehouses, stock records, purchase orders, and order items.
- To enforce data integrity through primary keys, foreign keys, unique constraints, validation rules, and referential integrity.
- To provide role-based access for administrators, inventory managers, and warehouse staff.
- To provide a responsive and user-friendly interface for performing inventory management tasks.

6. System Design (ERD)

6.1 Entity Identification

Strong Entities

1. **Product**
   - Stores product information.
   - Acts as the main entity for inventory tracking.
2. **Supplier**
   - Stores supplier and vendor information.
   - Operates independently of other entities.
3. **Warehouse**
   - Stores warehouse and storage location information.
   - Operates independently of other entities.
4. **Category**
   - Stores product categories.
   - Organizes products into groups.
5. **User**
   - Stores system user information.
   - Supports user roles and warehouse management.

Associative Entities

1. **Stock**
   - Associates a product with a warehouse.
   - Tracks the quantity of a product in a warehouse.
   - Represents the relationship between Product and Warehouse.
2. **PurchaseOrder**
   - Associates a supplier and warehouse with a purchasing transaction.
   - Stores order details such as order number, dates, status, and total amount.
3. **OrderItem**
   - Associates a purchase order with a product.
   - Stores the quantity, unit price, and subtotal for each ordered product.

6.2 Entity Attributes

Product

Attribute	Data Type	Key	Description
id	INT	PK	Unique product identifier
name	VARCHAR(255)	-	Product name
sku	VARCHAR(50)	UNIQUE	Stock Keeping Unit
description	TEXT	-	Product description
price	DECIMAL(10,2)	-	Unit price
category_id	INT	FK	Reference to Category
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

Supplier

Attribute	Data Type	Key	Description
id	INT	PK	Unique supplier identifier
name	VARCHAR(255)	-	Supplier name
contact_person	VARCHAR(255)	-	Contact person
email	VARCHAR(255)	-	Email address
phone	VARCHAR(20)	-	Phone number
address	TEXT	-	Physical address
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

Warehouse

Attribute	Data Type	Key	Description
id	INT	PK	Unique warehouse identifier
name	VARCHAR(255)	-	Warehouse name

Attribute	Data Type	Key	Description
location	VARCHAR(255)	-	Physical location
capacity	INT	-	Maximum storage capacity
manager_id	INT	FK	Reference to User
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

Category

Attribute	Data Type	Key	Description
id	INT	PK	Unique category identifier
name	VARCHAR(255)	-	Category name
description	TEXT	-	Category description
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

User

Attribute	Data Type	Key	Description
id	INT	PK	Unique user identifier
name	VARCHAR(255)	-	User name
email	VARCHAR(255)	UNIQUE	Email address
password	VARCHAR(255)	-	Encrypted password
role	ENUM	-	User role
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

Stock

Attribute	Data Type	Key	Description
id	INT	PK	Unique stock record identifier
product_id	INT	FK	Reference to Product
warehouse_id	INT	FK	Reference to Warehouse
quantity	INT	-	Current stock quantity
reorder_level	INT	-	Minimum stock before reorder
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

PurchaseOrder

Attribute	Data Type	Key	Description
id	INT	PK	Unique purchase order identifier
order_number	VARCHAR(50)	UNIQUE	Order reference
supplier_id	INT	FK	Reference to Supplier
warehouse_id	INT	FK	Reference to Warehouse
order_date	DATE	-	Date order was placed
expected_date	DATE	-	Expected delivery date
status	ENUM	-	Order status
total_amount	DECIMAL(10,2)	-	Total order value
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

OrderItem

Attribute	Data Type	Key	Description
id	INT	PK	Unique order item identifier
purchase_order_id	INT	FK	Reference to PurchaseOrder
product_id	INT	FK	Reference to Product
quantity	INT	-	Quantity ordered
unit_price	DECIMAL(10,2)	-	Price per unit
subtotal	DECIMAL(10,2)	-	Line item total
created_at	TIMESTAMP	-	Creation timestamp
updated_at	TIMESTAMP	-	Update timestamp

6.3 Relationship and Cardinality Analysis

Binary Relationships

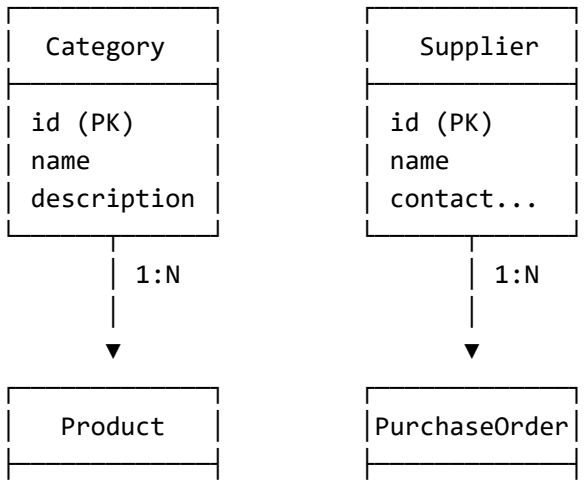
1. **Supplier - PurchaseOrder**
   - Relationship: A supplier places purchase orders.
   - Cardinality: One-to-many (1:N).
   - Participation: Partial.
   - Implementation: `supplier_id` in `PurchaseOrder`.
2. **Warehouse - PurchaseOrder**
   - Relationship: A warehouse receives purchase orders.
   - Cardinality: One-to-many (1:N).
   - Participation: Partial.
   - Implementation: `warehouse_id` in `PurchaseOrder`.
3. **Category - Product**
   - Relationship: A category contains products.
   - Cardinality: One-to-many (1:N).
   - Participation: Partial.
   - Implementation: `category_id` in `Product`.

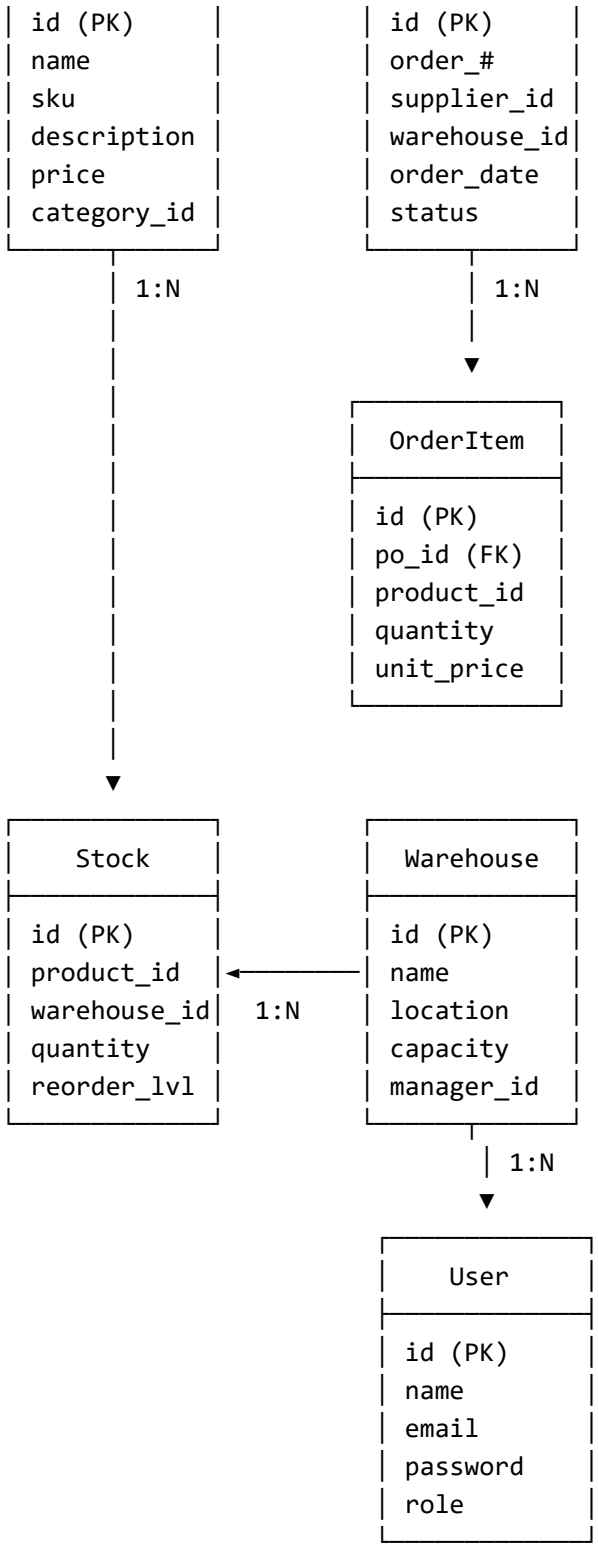
4. **Product - Stock**
    - Relationship: A product has stock records.
    - Cardinality: One-to-many (1:N).
    - Participation: Total.
    - Implementation: `product_id` in `Stock`.
5. **Warehouse - Stock**
    - Relationship: A warehouse stores stock records.
    - Cardinality: One-to-many (1:N).
    - Participation: Total.
    - Implementation: `warehouse_id` in `Stock`.
6. **Product - OrderItem**
    - Relationship: A product appears in order items.
    - Cardinality: One-to-many (1:N).
    - Participation: Partial.
    - Implementation: `product_id` in `OrderItem`.
7. **PurchaseOrder - OrderItem**
    - Relationship: A purchase order contains order items.
    - Cardinality: One-to-many (1:N).
    - Participation: Total.
    - Implementation: `purchase_order_id` in `OrderItem`.
8. **User - Warehouse**
    - Relationship: A user manages a warehouse.
    - Cardinality: One-to-many (1:N).
    - Participation: Partial.
    - Implementation: `manager_id` in `Warehouse`.

Many-to-Many Relationships

1. **Product - Warehouse**
    - A product can be stored in multiple warehouses.
    - A warehouse can store multiple products.
    - Resolved through the `Stock` entity.
2. **Supplier - Product**
    - A supplier can supply multiple products.
    - A product can be supplied by multiple suppliers.
    - Represented through `PurchaseOrder` and `OrderItem`.

6.4 ERD Diagram





7. ERD to Database

7.1 Entity-to-Table Mapping

Each entity in the ERD is converted into a database table.

ERD Entity	Database Table	Primary Key	Purpose
User	users	id	Stores system users and roles
Category	categories	id	Stores product categories
Supplier	suppliers	id	Stores supplier information
Warehouse	warehouses	id	Stores warehouse locations and managers
Product	products	id	Stores product information
Stock	stocks	id	Tracks product quantities by warehouse
PurchaseOrder	purchase_orders	id	Stores purchase order information
OrderItem	order_items	id	Stores products and quantities in purchase orders

7.2 Database Tables and Attributes

users

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique user identifier
name	VARCHAR	Required	User name
email	VARCHAR	Required, unique	User email address
email_verified_at	TIMESTAMP	Nullable	Email verification timestamp
password	VARCHAR	Required	Hashed user password
role	ENUM	Required, default `staff`	User role
remember_token	VARCHAR	Nullable	Password reset token
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

categories

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique category identifier
name	VARCHAR	Required	Category name
description	TEXT	Nullable	Category description
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

suppliers

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique supplier identifier
name	VARCHAR	Required	Supplier name
contact_person	VARCHAR	Nullable	Contact person name
email	VARCHAR	Nullable	Supplier email address
phone	VARCHAR(20)	Nullable	Supplier phone number
address	TEXT	Nullable	Supplier address
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

warehouses

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique warehouse identifier
name	VARCHAR	Required	Warehouse name
location	VARCHAR	Required	Warehouse location
capacity	INTEGER	Required, default `0`	Maximum storage capacity
manager_id	BIGINT UNSIGNED	Foreign key to `users.id`, nullable	Assigned warehouse manager
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

products

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique product identifier
name	VARCHAR	Required	Product name
sku	VARCHAR(50)	Required, unique	Stock Keeping Unit
description	TEXT	Nullable	Product description
price	DECIMAL(10,2)	Required	Product unit price
category_id	BIGINT UNSIGNED	Foreign key to `categories.id`, nullable	Product category
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

stocks

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique stock record identifier
product_id	BIGINT UNSIGNED	Foreign key to `products.id`, required	Product in stock
warehouse_id	BIGINT UNSIGNED	Foreign key to `warehouses.id`, required	Warehouse containing stock
quantity	INTEGER	Required, default `0`	Current stock quantity
reorder_level	INTEGER	Required, default `10`	Minimum quantity before reorder
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp
product_id + warehouse_id	Composite	Unique constraint	Prevents duplicate stock records

purchase_orders

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique purchase order identifier
order_number	VARCHAR(50)	Required, unique	Purchase order reference number
supplier_id	BIGINT UNSIGNED	Foreign key to `suppliers.id`, required	Supplier receiving the order

Column	Data Type	Constraints	Description
warehouse_id	BIGINT UNSIGNED	Foreign key to `warehouses.id`, required	Warehouse receiving the order
order_date	DATE	Required	Date the order was placed
expected_date	DATE	Nullable	Expected delivery date
status	ENUM	Required, default `pending`	Order status: `pending`, `received`, or `cancelled`
total_amount	DECIMAL(10,2)	Required, default `0`	Total purchase order amount
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

order_items

Column	Data Type	Constraints	Description
id	BIGINT UNSIGNED	Primary key, auto-increment	Unique order item identifier
purchase_order_id	BIGINT UNSIGNED	Foreign key to `purchase_orders.id`, required	Related purchase order
product_id	BIGINT UNSIGNED	Foreign key to `products.id`, required	Ordered product
quantity	INTEGER	Required	Quantity ordered
unit_price	DECIMAL(10,2)	Required	Price per unit
subtotal	DECIMAL(10,2)	Required	Total price for the order item
created_at	TIMESTAMP	Nullable	Creation timestamp
updated_at	TIMESTAMP	Nullable	Update timestamp

7.3 Relationship Implementation

ERD Relationship	Database Implementation
Category to Product	`products.category_id` references `categories.id`
User to Warehouse	`warehouses.manager_id` references `users.id`
Supplier to PurchaseOrder	`purchase_orders.supplier_id` references `suppliers.id`

ERD Relationship	Database Implementation
Warehouse to PurchaseOrder	`purchase_orders.warehouse_id` references `warehouses.id`
Product to Stock	`stocks.product_id` references `products.id`
Warehouse to Stock	`stocks.warehouse_id` references `warehouses.id`
PurchaseOrder to OrderItem	`order_items.purchase_order_id` references `purchase_orders.id`
Product to OrderItem	`order_items.product_id` references `products.id`

7.4 Many-to-Many Relationship Resolution

The many-to-many relationship between products and warehouses is represented by the `stocks` table. Each stock record links one product to one warehouse and stores the quantity and reorder level for that combination. The unique constraint on `(product_id, warehouse_id)` prevents duplicate records.

The relationship between suppliers, products, and purchase orders is represented through the `purchase_orders` and `order_items` tables. A purchase order links a supplier and warehouse, while each order item links a product to that purchase order with its quantity, unit price, and subtotal.

7.5 Referential Integrity

Foreign key constraints maintain relationships between tables and help prevent invalid records. The following integrity rules apply:

- A product may reference an existing category.
- A warehouse may reference an existing user as its manager.
- A purchase order must reference an existing supplier and warehouse.
- A stock record must reference an existing product and warehouse.
- An order item must reference an existing purchase order and product.
- The combination of `product_id` and `warehouse_id` in the `stocks` table must be unique.
- The product SKU in the `products` table must be unique.
- The order number in the `purchase_orders` table must be unique.
- The user email in the `users` table must be unique.

These constraints help maintain accurate relationships and consistent inventory, supplier, warehouse, and purchasing data.

8. Conclusion

The Stock Management System is designed to provide a centralized and organized approach to managing inventory across multiple warehouses. The system addresses common inventory-management problems by maintaining product, supplier, warehouse, stock, purchase order, and order item information in a relational database.

The ERD identifies the major entities and their relationships, including one-to-many relationships and many-to-many relationships resolved through associative entities. The database design converts these entities into tables with appropriate primary keys, foreign keys, unique constraints, and referential integrity rules.

The proposed design supports accurate stock monitoring, supplier and purchase order tracking, warehouse management, and better access to inventory information. Overall, the system provides a structured foundation for efficient inventory management and improved decision-making.